

PLANOS DE TESTES

Cidade Virtual - PI - III

Avaliação do Módulo 2 - Testes de Software Aplicado

Maio de 2026

Nome Completo:	Diego Serafim de Sousa
Disciplina:	Projeto Integrador de Tecnologia da Informação III
Semestre:	2026/1
Curso:	Tecnologia da Informação
Público-alvo:	Moradores, comerciantes e autônomos.
Local de realização:	Cabanagem, Belém, Pará

1 TEMA DA AÇÃO

PLANO DE TESTES DA SISTEMA CIDADE VIRTUAL - PLATAFORMA WEB PARA CONEXÃO COMUNITÁRIA

2 RESUMO

Este documento apresenta o Plano de Testes do sistema Cidade Virtual, uma plataforma web desenvolvida para conectar moradores, comerciantes e autônomos do bairro e da cidade. Foram definidas estratégias de testes unitários, integração, sistema (E2E), segurança e carga, utilizando as ferramentas Jest, Supertest, Cypress, OWASP ZAP e K6. O plano estabelece objetivos, escopo, cronograma, gestão de erros e riscos, visando garantir a qualidade, confiabilidade e segurança da aplicação antes de sua ampla divulgação à comunidade.

Palavras-chave: Testes de software. Qualidade. Cidade Virtual. MERN. Tecnologia social.

3 INTRODUÇÃO

O projeto Cidade Virtual vem sendo desenvolvido desde 2022 pelo estudante Diego Serafim de Sousa, sendo continuamente aprimorado no âmbito acadêmico das disciplinas de Projeto Integrador da UFMS. Trata-se de uma plataforma web baseada na pilha MERN (MongoDB, Express.js, React.js, Node.js) que tem como objetivo conectar os moradores, comerciantes e trabalhadores autônomos do bairro, promovendo o fortalecimento da economia local e a integração da comunidade por meio da tecnologia.

3.1 Objetivo do Plano de Testes

O Plano de Testes tem como objetivo geral estabelecer uma estratégia sistemática para a verificação dinâmica do sistema Cidade Virtual, garantindo que:

- As funcionalidades atendam aos requisitos específicos;
- O sistema seja confiável, seguro e responsivo;
- Defeitos sejam identificados precocemente, reduzindo custos de correção;
- A comunidade receba um produto de qualidade.

3.2 Importância do plano

A aplicação de testes em projetos de extensão comunitária não é apenas para questões técnicas, mas um compromisso ético com os usuários. Um sistema com falhas pode gerar desconfiança, prejuízos e abandono da ferramenta. Este plano busca mitigar tais riscos, alinhando-se aos princípios de Verificação, Validação e Testes (VVT).

3.3 Escopo do projeto a ser testado

Serão considerados para testes os componentes do sistema Cidade Virtual:

Tabela 1 – Escopo do projeto Cidade Virtual e prioridades de teste

Componente	Descrição	Prioridade
Cadastro e autenticação de usuários	Registro de moradores, comerciantes e autônomos; login com JWT; perfis diferenciados	Crítica
Busca por comércios e serviços	Filtros por categoria, proximidade geográfica, avaliação; ordenação	Crítica
Sistema de avaliações	Usuários avaliam comércios/serviços; cálculo de média; listagem de avaliações	Alta
Mural de eventos e campanhas	Criação, edição, exclusão e listagem de eventos comunitários; doações	Média
Perfil do comerciante/autônomo	Edição de informações, horários de funcionamento, upload de fotos	Média
Área administrativa	Gestão de usuários, aprovação de comércios, relatórios básicos	Baixa

Fonte: Elaborado pelo autor (2026)

Não estão no escopo inicial deste plano: Testes de compatibilidade entre navegadores antigos (priorização do Chrome e Firefox atuais), testes de acessibilidade completos e testes de internacionalização.

3.4 Recursos do projeto considerados

Os artefatos e recursos serão utilizados como base para a construção dos testes:

- Documento de requisitos funcionais e não funcionais (PI-I);
- Diagrama de Entidade-Relacionamento (DER);
- Contratos da API (OpenAPI/Swagger).
- Protótipo de interface;
- Código-fonte disponível em repositório GitHub (branch develop);
- Ambiente de homologação: <https://cidadevirtual.vercel.app/>

4 ESTRATÉGIA DE TESTE

4.1 Visão geral das estratégias

Serão empregados quatro níveis de teste, alinhados à pirâmide de testes (Cohn, 2009), respeitando a proporção recomendada para projetos individuais: 70% unitários, 20% integração, 10% sistema (E2E). E serão incluídos os testes de segurança de carga como camadas complementares.

4.2 Testes Unitários

Tabela 2 – Especificação e planejamento dos testes unitários

Aspecto	Descrição
Objetivos	Validar o comportamento isolado de funções, métodos e classes; garantir a correção de regras de negócio; atingir cobertura mínima de 70% das linhas críticas
Escopo	Services (regras de negócio), Utils (funções auxiliares), Models (validação de esquema), Controllers (lógica de requisição)
Ferramentas	Jest (framework de testes all-in-one), Supertest (para testes de controllers)
Técnicas	Partição por classes de equivalência, análise de valor limite, injeção de dependências com mocks
Exemplo de caso de teste	Verificar se a função <code>calcularAvaliacaoMedia([5, 4, 3])</code> retorna exatamente 4.0 e se trata corretamente lista vazia retornando 0

Fonte: Elaborado pelo autor (2026)

4.3 Testes de Integração

Tabela 3 – Especificação e planejamento dos testes de integração

Aspecto	Descrição
Objetivos	Verificar a comunicação entre componentes (API e banco de dados, serviços internos); validar contratos de interface
Escopo	Endpoints de API REST (CRUD de usuários, comércios, avaliações, eventos); integração com MongoDB (operações de leitura/escrita)
Ferramentas	Jest, Supertest (para chamadas HTTP reais), banco de dados de teste (MongoDB em memória ou container Docker)
Estratégia	Top-down: começar pelos endpoints principais (POST /api/auth/register, GET /api/comercios) e descer para os níveis inferiores
Exemplo de caso de teste	Enviar requisição POST /api/comercios com dados válidos e verificar se o banco retorna o documento criado com _id gerado

Fonte: Elaborado pelo autor (2026)

4.4 Testes de Sistema (End-to-End)

Tabela 4 – Especificação e planejamento dos testes de sistema (E2E)

Aspecto	Descrição
Objetivos	Simular o comportamento real do usuário final navegando pela aplicação; validar fluxos completos (front-end, back-end, banco de dados)
Escopo	Fluxos críticos: 1. cadastro de comerciante→login→cadastro de produtos; 2. morador→busca por categoria→avaliação; 3. autônomo→criação de evento
Ferramentas	Cypress (automação de navegador)
Frequência	Executadas antes de cada release (a cada 2 semanas) e no pipeline de CI/CD
Exemplo de caso de teste	Cenário: Morador acessa a 'home', realiza uma busca por 'doceiras', clica no primeiro resultado, avalia com nota 5 e verifica se a média do comércio foi atualizada

Fonte: Elaborado pelo autor (2026)

4.5 Testes de Segurança

Tabela 5 – Especificação e planejamento dos testes de segurança

Aspecto	Descrição
Objetivos	Identificar vulnerabilidades comuns em aplicações web; injeção de SQL, XSS (Cross-Site-Scripting), autenticação quebrada, JWT exposto
Escopo	Endpoints de autenticação (/api/auth); campos de entrada que aceitam texto livre (busca, avaliação, descrição de comércio)
Ferramentas	OWASP ZAP (análise estática e dinâmica), Snyk (verificação de dependências vulneráveis)
Exemplo de caso de teste	Tentar inserir script <code><script>alert('XSS')</script></code> no campo de busca e verificar se o sistema sanitiza a saída

Fonte: Elaborado pelo autor (2026)

4.6 Testes de Carga e Stress

Tabela 6 – Especificação e planejamento dos testes de carga e stress

Aspecto	Descrição
Objetivos	Avaliar o comportamento do sistema sob alta demanda simultânea; identificar gargalos de desempenho e limites de escalabilidade
Escopo	Endpoint de busca (GET /api/comercios?categoria=alimentos), por ser o mais acessado; endpoint de cadastro de avaliação (POST /api/avaliacoes)
Ferramentas	K6 (teste de carga open-source)
Métrica	O sistema deve suportar 50 usuários simultâneos com tempo de resposta < 2 segundos no endpoint de busca
Exemplo de caso de teste	Simular 100 usuários virtuais executando buscas simultâneas por 5 minutos; medir tempo de resposta médio e taxa de erro

Fonte: Elaborado pelo autor (2026)

4.7 Testes de Usabilidade e Aceitação

Tabela 7 – Especificação e planejamento dos testes de usabilidade e aceitação

Aspecto	Descrição
Objetivos	Validar se o sistema atende às necessidades reais dos usuários (moradores e comerciantes) e se é fácil de usar por pessoas com diferentes níveis de familiaridade digital
Escopo	Telas principais: cadastro, busca, perfil do comércio, mural de eventos
Participantes	Lucia Matos (comerciante) e Roseane Soares (moradora), representantes da comunidade
Procedimento	Sessão de 1 hora com cada participante, realizando tarefas pré-definidas enquanto o desenvolvedor observa e registra dificuldades
Critério de sucesso	100% das tarefas concluídas sem auxílio do moderador

Fonte: Elaborado pelo autor (2026)

5 RECURSOS E AMBIENTES

5.1 Recursos de hardware

Tabela 8 – Especificações de hardware e suas finalidades

Recurso	Especificação	Finalidade
Notebook	Processador Intel Core i5 ou superior, 16GB RAM, 256GB SSD	Desenvolvimento e execução dos testes
Celular (Android/iOS)	Navegador Chrome ou Firefox	Testes de usabilidade e responsividade
Conexão com a internet	Mínimo 10 Mbps	Acesso a repositórios, execução de pipelines CI/CD

Fonte: Elaborado pelo autor (2026)

5.2 Dependências de software

Tabela 9 – Dependências de software, versões e finalidades

Software	Versão	Finalidade
Node.js	18.x ou superior	Execução do back-end e dos testes unitários
MongoDB	6.x ou superior (local ou Atlas)	Banco de dados de desenvolvimento e teste
Docker (opcional)	20.x	Para subir ambiente de teste isolado (MongoDB e API)
Jest	^29.0	Framework de testes unitários e de integração
Supertest	^6.0	Teste de API HTTP
Cypress	^13.0	Testes end-to-end
K6	^0.46	Testes de carga
OWASP ZAP	2.14	Testes de segurança
Git	2.x	Controle de versão e integração com GitHub

Fonte: Elaborado pelo autor (2026)

5.3 Equipe e responsabilidades

Considerando o caráter individual do projeto, as responsabilidades são atribuídas ao estudante desenvolvedor Diego Serafim de Sousa, que acumula os papéis de:

Tabela 10 – Atribuição de papéis e responsabilidades da equipe

Papel	Responsabilidade
Autor e inspetor único	Criação e execução de todos os testes; registro e correção de defeitos; geração de relatórios de cobertura
Moderador	Planejamento e coordenação das atividades de teste; garantia do cumprimento do cronograma
Analista de qualidade	Definição das estratégias; análise de resultados; priorização de defeitos

Fonte: Elaborado pelo autor (2026)

Participação externa (apenas para testes de aceitação e usabilidade):

- Lucia Matos (comerciante local): validação da usabilidade e adequação das funcionalidades ao pequeno negócio
- Roseane Soares (Morador local): validação da experiência do usuário final.

6 GESTÃO DE ERROS E DEFEITOS

6.1 Registro e acompanhamento

A priorização dos defeitos será feita com base na matriz de severidade x impacto:

Tabela 11 – Campos para registro e acompanhamento de defeitos

Campo	Descrição
ID	Identificador único (ex.: BUG-001)
Data	Data da descoberta
Artefato	Componente afetado (ex.: API, front-end, banco)
Descrição	Detalhamento do problema
Severidade	Crítico / Maior / Menor
Prioridade	Alta / Média / Baixa
Status	Aberto / Em correção / Corrigido / Fechado
Solução	Breve descrição da correção aplicada

Fonte: Elaborado pelo autor (2026)

6.2 Política de priorização

A priorização dos defeitos será feita com base na matriz de severidade x impacto:

Tabela 11 – Campos para registro e acompanhamento de defeitos

Severidade / Impacto	Alto	Médio	Baixo
Crítico	Prioridade 1 (corrigir imediatamente)	Prioridade 1 (corrigir imediatamente)	Prioridade 2 (corrigir dentro de 48h)
Maior	Prioridade 2 (corrigir dentro de 48h)	Prioridade 2 (corrigir dentro de 48h)	Prioridade 3 (corrigir na próxima sprint)
Menor	Prioridade 3 (corrigir na próxima sprint)	Prioridade 3 (corrigir na próxima sprint)	Prioridade 4 (backlog, corrigir quando possível)

Fonte: Elaborado pelo autor (2026)

Ex. prático: Defeito que impede o cadastro de novos comerciantes (severidade crítica, impacto alto) recebe prioridade 1 e deve ser corrigido antes de qualquer outro trabalho.

6.3 Ciclo de vida do defeito

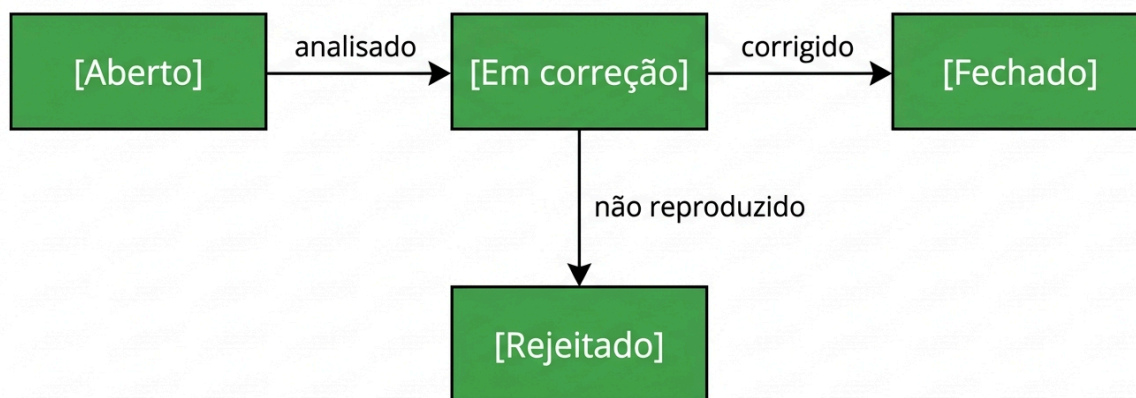


Figura 1 – Fluxo do ciclo de vida de um defeito
Fonte: Elaborado pelo autor (2026)

O desenvolvedor é responsável por avançar o defeito pelos estágios e registrar a solução aplicada.

7 CRONOGRAMA

O desenvolvimento dos testes será realizado ao longo de 4 semanas, conforme a tabela abaixo.

Tabela 12 – Cronograma de implementação dos testes e estimativa

Semana	Atividade	Tipos teste	de	Esforço estimado
Semana 1 (04-10/05/2026)	Configuração do ambiente de testes; implementação de testes unitários para serviços e utilitários	Unitários		10h
Semana 2 (11-17/05/2026)	Finalização dos testes unitários (controllers, models); início dos testes de integração (endpoints principais)	Unitários, Integração		12h
Semana 3 (18-24/05/2026)	Testes de integração (endpoints secundários); configuração e implementação dos testes E2E com Cypress	Integração, Sistema (E2E)		10h

Semana 4 (25-31/05/2026)	Testes de segurança (OWASP ZAP) e carga (K6); testes de aceitação com representantes da comunidade; correção de defeitos remanescentes	Segurança, Carga, Aceitação	8h
Total			40h

Fonte: Elaborado pelo autor (2026)

Observações:

- As atividades de correção de defeitos são contínuas e estão incluídas nas horas de cada semana.
- Os testes de usabilidade e aceitação com Lúcia Matos e Roseane Soares serão agendados conforme disponibilidade delas, previstos para a semana 4.

8 RISCOS E MITIGAÇÕES

Tabela 13 – Matriz de riscos, impactos e estratégias de mitigação

ID	Risco	Probabilidade	Impacto	Estratégia de mitigação
R01	Atraso na entrega do cronograma devido à complexidade não prevista	Média	Médio	Priorizar testes críticos nas primeiras semanas; deixar testes menos importantes para o final; revisar o plano semanalmente
R02	Indisponibilidade dos representantes da comunidade (Lúcia Matos, Roseane Soares) para testes de aceitação	Média	Alto	Agendar com pelo menos 7 dias de antecedência; ter um plano alternativo (teste remoto via gravação de tela)
R03	Falta de cobertura adequada de testes devido a código legado não estável	Alta	Alto	Refatorar pequenas partes do código antes de testar (priorizar componentes de maior impacto)
R04	Ambiente de homologação (Vercel) indisponível durante testes E2E	Baixa	Médio	Manter um ambiente de teste local (Docker, MongoDB) com fallback
R05	Dificuldade em configurar ferramentas de teste (Cypress, K6, OWASP ZAP)	Média	Baixo	Seguir documentação oficial; buscar suporte nos fóruns das ferramentas; alocar 4h extras para configuração

R06	Falsos positivos/negativos em testes de segurança automatizados	Média	Médio	Validar manualmente os alertas do OWASP ZAP; não confiar cegamente na automação
-----	---	-------	-------	---

Fonte: Elaborado pelo autor (2026)

REFERÊNCIAS

COHN, Mike. **Succeeding with Agile: Software Development Using Scrum**. Addison-Wesley, 2009.

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. **Introdução ao teste de software**. Rio de Janeiro: Gen Ltc, 2016. ISBN 9788595155732.

GONÇALVES, Priscila de Fátima et al. **Testes de software e gerência de configuração**. Porto Alegre: Sagah, 2019. ISBN 9788595029361.

MYERS, Glenford J.; SANDLER, Corey; BADGETT, Tom. **The Art of Software Testing**. 3. ed. Wiley, 2011.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011. ISBN 9788579361081.

VALENTE, Marco Tulio. **Engenharia de Software moderna: princípios e práticas para desenvolvimento de software com produtividade**. v.1, n.24, 2020. Disponível em: <https://link.ufms.br/bljNp>. Acesso em 20 dez de 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação - Referências - Elaboração. Rio de Janeiro: ABNT, 2018.